

LikeC4 и AsciiDoc в проектировании систем

LikeC4 и AsciiDoc для проектирования систем

Архитектура как код · моделирование · документация

Демонстрация возможностей

2. Архитектура как код

Идея **Architecture as Code (AaC)**: архитектурные решения и ограничения описаны в **машинно-читаемых** артефактах, **версионизируются** в репозитории и **проверяются** в CI/CD. Замысел становится **валидируемым**, а не только текстом в Confluence.

Ключевой сдвиг: от «рекомендаций в документах» к **проверяемым утверждениям**, которые можно сопоставить с фактической структурой системы.

3. Откуда берётся практика

- **Infrastructure as Code** — инфраструктура как код с теми же принципами версий и воспроизводимости.
- **Policy as Code** — формальные правила безопасности и соответствия в pipeline.
- **Документация как артефакты** — диаграммы и правила в текстовом виде, удобном для ревью.
- **Governance на платформе** — единые гарантии для многих команд и сервисов.

4. Как это работает (четыре опоры)

1. **Артефакты хранятся как код** — ADR/HLD/LLD/DD, схемы зависимостей, контракты API и событий, ограничения на границы модулей, параметры устойчивости в структурированном виде.
2. **Ограничения превращаются в правила** — циклы, контракты, политики на публичные интерфейсы, качество шаблонов ADR.
3. **Проверки в CI/CD** — gate в pipeline: fail merge или отчёт для архитектурного ревью.
4. **Трассируемость** — связь требований, решений, кода, контрактов и эксплуатации (метрики, логи).

5. LikeC4: зачем

LikeC4 — язык и инструмент для **модели C4** в виде кода: **контекст, контейнеры, компоненты**, при необходимости **deployment**.

- Диаграммы и связи — **текстом** (`*.c4`), дифф в Git, ревью как у кода.
- **Представления (views)** — какие элементы и связи показать на схеме; заголовки и описания для публикации.
- **Сборка** — статический веб-интерфейс, **экспорт** (например, PlantUML), встроенная навигация по модели.
- **Проверки** — тесты и линтер на консистентность модели (в проекте — Vitest, `npm run test:likec4`).

6. Пример: контекст (LikeC4)

Граница системы и внешние акторы — уровень C4 **System context**.

[Контекстная диаграмма LikeC4 на GitHub Pages](#)

7. Пример: код (LikeC4)

Фрагменты из `likec4/model.c4` и `likec4/views.c4`: объявление **контейнера** и **базы данных**, **связь** между элементами, выборка на диаграмму через `view`.

7.1. Контейнер (микросервис)

```
user-service = container 'User Service' {  
    summary 'Профили, сессии, роли (RBAC)...'  
    technology '[Spring Boot]'  
    icon tech:java  
}
```

7.2. База данных

```
postgres-master = database 'PostgreSQL (master)' {  
    technology '[PostgreSQL]'  
    summary 'Единая точка записи: users_db, requests_db, ...'  
    icon tech:postgresql  
}
```

7.3. Связь (отношение)

Протокол, подпись к линии и теги `#sync` / `#async`:

```
user-service -[tcp]-> postgres-master 'Запись: users, sessions, roles' 'TCP, JDBC, PostgreSQL' { #sync }  
user-service -[tcp]-> cache 'Профили, сессии (Cache-Aside)' 'TCP, Redis RESP' { #sync }
```

7.4. Представление: контекст (system)

```
view context of system {  
    title 'System/Контекстная диаграмма'  
    description 'Диаграмма контекста системы'  
  
    include users,  
            system,  
            system.*,  
            system_ext  
}
```

7.5. Представление: контейнерный view (backend)

```
view backend_container of system.backend {  
    title 'System/Обработка пользовательских запросов/Контейнерная диаграмма'  
    description 'Микросервисы, взаимодействие с БД, кэшем, Event Bus...'  
  
    include system,  
            backend,  
            backend.*,  
            *  
}
```

7.6. Представление: deployment

```
deployment view deployment_production {  
    title 'System/Развёртывание/Production'  
    description 'Edge: WAF/CDN. Регион A: VM + K8s...'
```



```
include production.**,  
  production  
}
```

Рендер: `npm run build:likes4` → статический сайт; на странице view — диаграмма и навигация по модели.

8. AsciiDoc: зачем рядом с моделью

AsciiDoc — разметка для длинной документации: требования, расчёты НФТ и т.д.

- Один источник: **разделы, перекрёстные ссылки**, условные включения, единый стиль экспорта.
- **PDF / HTML** — отчёты и отчётность внутри команды (**asciidocctor-pdf**, GitHub/GitLab, Confluence).
- **Связь с артефактами** — ссылки на **openapi/**, **dbml/**, **likec4/**, **infra/**; картинки из **doc/images** (экспорт диаграмм и DBML в PNG).

Текстовые документы дополняют **графическую модель**: не заменяют её, а фиксируют обоснование и числа.

9. Связка в репозитории

- Каталог `likec4/` — модель, deployment, `views`; `npm run build:likec4` → статический сайт; публикация через GitHub Actions (Pages).
- Каталог `doc/` — методичка в AsciiDoc, иллюстрации из тех же схем.
- Единый `pull request`: изменения модели и документа видны вместе.

9.1. Какие возможности появляются?

- Контроль в CI/CD сравнения текущего описания модели и ее реализации.
- Написание разнообразных аналитических и фитнес функций.
- Генерация кода из модели.
- Подход `arch-first`.
- Единая система хранения метаданных.
- И дальше Вы ограничиваетесь только своей фантазией и ресурсами.

10. Итог

- АаС — версии, проверки, трассируемость; LikeC4 даёт **диаграммы как код**, AsciiDoc — **спецификацию и отчёт как код**.
- Для демонстрации достаточно: открыть `likec4` в IDE, собрать сайт, пройти тесты модели, собрать PDF из `doc/` или `presentations/`.

11. Вопросы

Обсуждение

12. Спасибо

Спасибо за внимание!